



MANUAL TEST GUIDE · MODULE 06 OF 11

Shipments

Functional-Flow Test Walkthroughs

Scenario-driven, hands-on QA guide for the **logistics.shipments** operational backbone — booking → shipment carry-forward, the ten-condition readiness gate, multi-leg journeys, and master/house consolidation. Every step traced to its BRD requirement ID.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives you a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

01 What Shipments does & how to read this guide

The **shipment** is the long-lived operational record of a cargo movement — the counterpart to the short-lived booking. It owns the journey, the parties, the cargo, the frozen charges, the routing, and the leg(s) that bind to physical equipment. One model, `logistics.shipment`; `transport_mode_id` is a field, not a subclass.

13

COMMANDS TO EXERCISE

3

TEST SCENARIOS

10

READINESS-GATE CHECKS

≥2

LEGS FOR MULTI-MODAL

The two state machines you will observe

Shipments carry a **governed lifecycle** (command-driven) plus an **operational projection** derived from movement events. Watch both on the form header.

```
status      : draft → created → pending_validation → ready_for_execution → in_execution | cancelled
op.status    : not_started → in_transit → arrived → delivered → closed (applies after in_execution)
validation: pending → passed | failed | override_approved
```

Two orthogonal classifications set at creation

MOVEMENT_STRUCTURE

- single_mode** — one mode end-to-end (demo FCL export)
- multimodal** — ≥2 legs, one carrier responsibility
- intermodal** — ≥2 legs, leg-wise responsibility

HIERARCHY_TYPE

- direct** — master only, no houses (demo)
- master / house / house_master_linked** — consolidation
- console** — grouped console master

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Direct FCL export	Booking → shipment carry-forward, readiness gate, guards (booking-not-ready, duplicate, amendment freeze, cancel)	<code>create_from_booking</code> , <code>mark_ready</code>	§02 · §03
B — Multi-leg journey	add/remove legs, the ≥2-leg gate for multimodal/intermodal, transshipment, per-leg detail	<code>add_leg</code> , <code>remove_leg</code>	§04
C — Master / house consolidation	House creation, link types & split-share reconcile, roll-up, HBL, POD & operational status	<code>create_house</code> , <code>link_house</code> , <code>capture_pod</code>	§05

Demo fixtures you start from (loaded by `--with-demo=all`)

Forwarder **Acme Forwarding** — **Mumbai**; customer **Globex Inc.**; carrier **Maersk Demo Line**; lane **INMUM** → **SGSIN**. The seeded shipment `CO-SEA-D-2026-000001` (2×40HC cotton garments, 18 t / 128 CBM, ocean freight buy 2400 / sell 3200 USD) sits in **created** from booking `demo_booking_globex_001` — Scenario A drives it forward from there.

02 Direct FCL export — carry-forward & readiness

Goal: take the confirmed Globex booking to a shipment that clears the completeness gate and reaches **ready_for_execution** with **no data re-keying**. **Role:** Operations clerk. **Start state:** booking `demo_booking_globex_001` is `confirmed`.

#	Action	Expected result	✓ Checkpoint	Trace
1	Open the confirmed booking; run Create Shipment (<code>create_from_booking</code>) choosing <code>movement_structure=single_mode</code> , <code>hierarchy_type=direct</code> .	New shipment created in created . Mode, org, POL/POD, place of receipt/delivery, incoterm, cut-offs, vessel/voyage and operation owner are copied from the booking .	Every routing/mode field matches the booking — nothing re-typed.	BTS-01 BTS-02
2	Read the Shipment / Job No. field.	A composed number is auto-stamped by the numbering engine, e.g. <code>CO-SEA-D-2026-000001</code> .	shipment_ref populated, not blank.	BTS-03
3	Confirm the single leg exists (sequence 1, sea) and one cargo line (2×40HC) is present.	Exactly one leg in <code>leg_status=planned</code> ; cargo line carried with commodity, package count, gross weight.	1 leg, 1 cargo line, both populated.	BTS-09
4	Confirm the mandatory parties (shipper, consignee, billing) + main-carriage carrier and the frozen freight charge are attached.	3 party roles present; charge shows buy 2400 / sell 3200 USD, frozen from the booking.	Parties + charge present; charge amounts read-only.	BTS-02
5	Run Mark Ready (<code>mark_ready</code>).	The ten-condition gate runs (see §03). All pass → status flips to ready_for_execution , <code>validation_state=passed</code> , <code>ready_at</code> stamped, and a <code>shipment.ready_for_execution</code> event fires.	Status = ready_for_execution; validation = passed.	BTS-19 BTS-21/22
6	Verify downstream consumers reacted (Documentation / Operations Execution / Tracking pick up the event).	A document checklist / execution plan / tracking activation appears for the shipment (covered in those modules' guides).	At least one downstream record now references this shipment.	event

✓ Scenario A pass criteria

The shipment reached **ready_for_execution** entirely from carried-forward data plus a clean readiness gate — zero fields re-keyed, and the number was system-composed.

⚠ Watch the mode nuance

The shipment-level `transport_mode_id` is the **commercial framing mode** (mode of the governing B/L) — for a multimodal journey it is *not* necessarily every leg's mode. Scenario B exercises this.

03 The readiness gate & the guards that must hold

Half of testing a backbone is confirming it **refuses** bad input. First, the ten-condition `mark_ready` gate; then four negative checks that each trip a specific guard.

The ten-condition completeness gate (BTS-19/21/22)

To force a failure, clear one required field on a fresh shipment before `mark_ready` — the command returns `ok:false` with a **reasons list**, status parks at **pending_validation**, and `validation_state=failed`.

#	Condition checked	Failure message you should see if missing
1	Source booking linked	"No source booking is linked"
2	Shipment number generated	"Shipment number not generated"
3	Movement structure selected	"Movement structure not selected"
4	Shipment hierarchy selected	"Shipment hierarchy not selected"
5	Operation owner assigned	"Operation owner not assigned"
6	Backup owner (only if <code>shipment.require_backup_owner</code> on)	"Backup owner required but not assigned"
7	Origin — facility <i>or</i> place of receipt	"Origin (facility or place of receipt) not set"
8	Destination — facility <i>or</i> place of delivery	"Destination (facility or place of delivery) not set"
9	Mandatory party roles present; cargo min detail + hazardous completeness	party / cargo-specific reason
10	Legs ≥ 2 for multimodal / intermodal (single_mode needs 1)	leg-count reason — see Scenario B

Override path

A manager may pass `override=true` (with `shipment.allow_validation_override` on) to push a failing shipment through — status still flips to ready but `validation_state=override_approved` and the reasons are retained for audit. Confirm the override is recorded, not silently discarded.

Negative checks — each must be refused

#	Action	Expected refusal	✓	Trace
7	<code>create_from_booking</code> against a cancelled booking.	Rejected: "cannot create a shipment from a cancelled booking".	Blocked	BTS-01
8	<code>create_from_booking</code> against a booking that is neither confirmed nor ready.	Rejected: booking status "is not creation-eligible; a confirmed/ready booking is required".	Blocked	BTS-01 / BR-01
9	Create a second shipment from a booking that already has one, <i>without</i> <code>allow_duplicate</code> .	Rejected: "a shipment already exists for this booking; pass <code>allow_duplicate</code> with a <code>split_reason</code> ".	Blocked	BTS-24 / BR-02
10	Edit any business field on a shipment in in_execution .	Rejected: "shipment is in execution — amendments are no longer permitted".	Blocked	BTS-23

*Controlled split: repeat step 9 with `allow_duplicate=true` + a `split_reason` — it should now succeed and stamp the split lineage. Cancel is exercised via `cancel` and lands the shipment in terminal **cancelled**.*

04 Multi-leg journeys — multimodal & intermodal

Goal: build a door-to-door journey out of sequenced legs and prove the engine treats a multi-mode shipment as **one record with many legs** — never a flat single record, and never per-mode duplicate models. **Start:** create a shipment with `movement_structure=multimodal` (or `intermodal`).

Leg 1 pickup (road) → Leg 2 main-carriage (sea) → Leg 3 transshipment (sea) → Leg 4 final delivery (road)
each leg: own mode · carrier · booking-ref · planned/actual dates · equipment.usage · leg_status

#	Action	Expected result	✓ Checkpoint	Trace
1	On the multimodal shipment, run <code>mark_ready</code> while it still has only one leg.	Gate fails — a leg-count reason is returned; status parks at pending_validation .	Single leg is rejected for multimodal.	gate #10 BR-LEG
2	Add legs with <code>add_leg</code> — a pre-carriage road leg and a main-carriage sea leg, each with its own carrier / booking-ref / planned dates / equipment usage.	Legs appear sequenced; each holds its own mode and provider; the shipment framing mode is unchanged.	≥2 legs, per-leg mode differs from framing mode.	BTS-09 BR-LEG
3	Add a transshipment leg and set the header <code>transshipment_id</code> .	Re-routing via the transship port is represented as its own leg; no data overwritten on the main leg.	Transship leg distinct from main carriage.	BR-LEG
4	Re-run <code>mark_ready</code> .	With ≥2 legs (and other conditions met) the gate now passes → ready_for_execution .	Multi-leg shipment reaches ready.	BTS-19
5	Attempt <code>remove_leg</code> without a reason.	Rejected: “remove_leg requires a reason”.	Reasonless removal blocked.	BR-SH-99
6	<code>remove_leg</code> with a reason, then re-check leg count vs. the structure.	Leg removed & audit-logged; if the count drops below 2 the shipment would fail a subsequent readiness re-check.	Removal audited; count enforced.	BR-SH-99

MULTIMODAL VS INTERMODAL	LEG OWNERSHIP (WITH MASTER/HOUSE)
• multimodal — ≥2 legs, one responsibility (single governing contract) • intermodal — ≥2 legs, leg-wise responsibility • Both require ≥2 legs at the readiness gate; <code>single_mode</code> requires exactly 1	• Shared main-carriage legs live on the master • First/last-mile legs live on the house • Each leg binds one <code>equipment.usage</code> — don't share a usage ref across legs unless intentionally consolidating

⚠ The anti-pattern to catch
If you ever see mode-specific shipment records (`shipment.sea` / `.air` / `.road`) or a single flat record trying to hold two modes, that is a defect — the design is **one** `logistics.shipment` + N `logistics.shipment.leg` rows.

05 Master / house consolidation, HBL & delivery

Goal: prove the master/house structure — houses linked to a master, roll-up totals, split-allocation reconcile — then close the loop with HBL and POD. **Start:** a `hierarchy_type=master` shipment plus one or more house shipments.

#	Action	Expected result	✓ Checkpoint	Trace
1	On the master, run <code>create_house</code> (or link an existing house) via <code>link_house</code> with a direct link type.	House ↔ master link row created; the house's <code>master_shipment_id</code> is set; a <code>house_master_linked</code> event fires.	House shows its master.	BTS-07/08
2	Add a second house as a split link with <code>allocation_share</code> (e.g. 0.6 then 0.5).	First split accepted; the second is rejected when the shares for a house sum > 1.0.	Over-allocation refused.	BR-SH-92
3	Trigger <code>recompute_consolidation</code> on the master (auto-fires on link/unlink).	Master <code>consolidation_total_weight</code> / <code>volume</code> / <code>packages</code> equal the sum across its linked houses.	Roll-up totals reconcile.	BTS-07
4	<code>unlink_house</code> one house (then again with <code>cancel=true</code>).	Link lifecycle → unlinked (or cancelled); house FK cleared; totals recomputed downward.	Unlink recomputes roll-up.	\$12
5	On a house, run <code>generate_hbl</code> ; on the master confirm the MBL slot.	HBL number stamped on the house; <code>bl_dependency_state</code> advances pending → referenced/issued.	HBL on house, MBL on master.	B/L slot
6	Drive execution, then <code>capture_pod</code> at last-mile; observe <code>set_operational_status</code> .	POD recorded; <code>operational_status</code> walks <code>not_started</code> → <code>in_transit</code> → <code>arrived</code> → <code>delivered</code> → <code>closed</code> . With <code>auto_close_on_pod</code> on, delivery auto-closes.	POD closes the shipment (or per config).	BTS-12

Config toggles worth flipping during the run

Config key	Default	Effect to verify
<code>shipment.auto_close_on_pod</code>	false	On → shipment closes automatically when POD captured + empty equipment returned.
<code>shipment.allow_partial_pod</code>	true	Permit POD on some houses while the master stays open.
<code>shipment.leg_status_derive</code>	event	event → leg status from movement events; manual → maintained by hand.
<code>shipment.allow_validation_override</code>	true	Gates the manager override path in §03.

✓ Full-module pass criteria

All three scenarios green: carry-forward + readiness (A), ≥2-leg multi-modal journey with audited leg edits (B), and master/house roll-up + split reconcile + HBL/POD close (C) — with every negative guard in §03–§04 confirmed to refuse.

Traceability: BTS-* IDs map to the “Booking to Shipment Creation” BRD; BR-LEG / BR-SH-* to `business-source/02-brs.md` (FR-G Legs & Multimodal). Cross-module effects (documents, execution, tracking, profitability) are verified in those modules' own test guides.